Shawareb's BoardGames Hub

Developer: Ahmed Amr Abdelbadea Ali Shawareb

Institution: Faculty of Computers and AI, Cairo University

---

1. My profiles and links

My LinkedIn : LinkedIn
My GitHub : GitHub
My Email : Email: eng.ahmedshawareb@gmail.com
Project GitHub Repo : Repo

---

2. Introduction

Shawareb's BoardGames Hub is a cross-platform terminal game engine developed in C++. While the initial scope of the project required a basic board game implementation, this framework was architected to support a dynamic roster of **11 distinct board games**, including classic Tic-Tac-Toe, Pyramic (SUS), Numerical Tic-Tac-Toe, Four-in-a-Row, a 5x5 variant, and a Dictionary-validated Word Game.

The project features a custom-built Text User Interface (TUI) utility for dynamic terminal rendering and a universal Artificial Intelligence engine powered by the Minimax algorithm with Alpha-Beta pruning, ensuring mathematically optimal gameplay against the computer.

Acknowledgment: The foundational abstract framework (`BoardGame_Classes.h`) was generously provided by Dr. Mohammad El-Ramly as part of the course materials. I made minor modifications to this base architecture to support the expanded roster of 11 games.

---

3. Object-Oriented Architecture

The framework is built entirely on the four main pillars of Object-Oriented Programming to ensure the engine is highly scalable and easy to maintain:

- **Abstraction:** The engine uses abstract base classes (`clsBoard`, `clsPlayer`) to define the universal concept of a board game, hiding the complex, low-level game mechanics from the main application controller.
- **Inheritance:** Every specific game variant (like Four-in-a-Row or the 5x5 game) inherits its core mechanics and attributes from the base classes. This drastically reduces code duplication and allows new games to be added rapidly.
- **Polymorphism:** The main game controller operates using runtime polymorphism. It manages the game loop using generic pointers, meaning it can execute `updateBoard()` or `isWin()` on any of the 11 games dynamically without needing to hardcode the rules for each specific game.
- **Encapsulation:** Sensitive game data, such as the dictionary hash table in the Word Game or the live grid states, are kept strictly private. The main application cannot alter the board directly and must interact with it safely through validated public methods.

---

4. Minimax Artificial Intelligence

Every game in the hub includes a "Smart AI" opponent powered by the **Minimax algorithm**, which mathematically calculates the optimal path to victory. To ensure the engine runs efficiently in real-time, two major optimizations were implemented:

- **Alpha-Beta Pruning:** Instantly cuts off (prunes) mathematically weak decision branches to save millions of calculations in larger games like Four-in-a-Row.
- **$O(1)$ Hash Tables:** For the complex Word Game variant, the English dictionary was loaded into an unordered_set, ensuring instant word validation and allowing the AI to evaluate board states without lagging.

---

5. Text User Interface (TUI)

Standard C++ output creates a basic, scrolling text feed, which isn't ideal for a playable game. To make the framework look and feel like a real application, a custom Text User Interface was implemented.

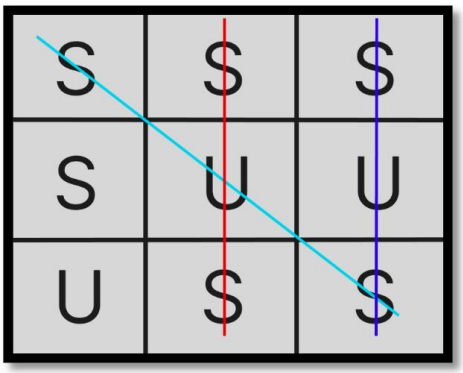
Since controlling terminal graphics in C++ is less about algorithmic problem-solving and more about executing highly specific Operating System commands and syntax, **I utilized AI tools to assist in generating the TUI components**. Using AI to handle this structural boilerplate allowed me to focus my engineering efforts on the complex OOP architecture and the AI logic. The resulting interface successfully provides smooth visuals, dynamic cursor movement, and real-time keyboard inputs across different operating systems.

---

6. Description of Games

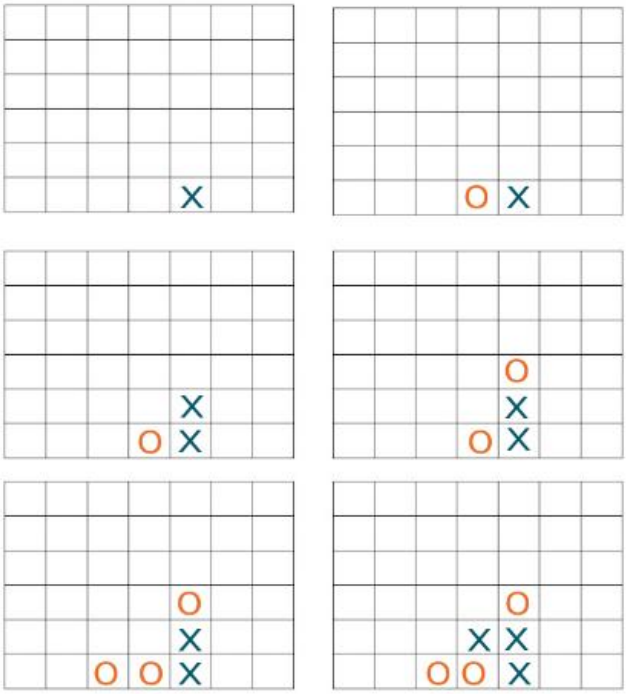
1. SUS

Introduction: A game on a 3x3 grid where the objective is to form the sequence "S-U-S".
Board: A 3x3 grid.
Rules: Players take turns placing either an "S" or a "U" in an empty square. A player must use the same letter each turn. A player scores a point for each "S-U-S" sequence they create.
Winning Condition: The game ends when the board is full. The player who has created the most "S-U-S" sequences wins.



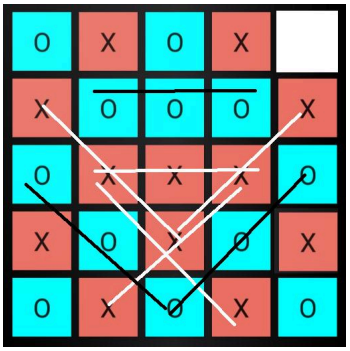
2. Four-in-a-row

Introduction: A 2D version of the classic game Connect Four where players mark a grid instead of dropping counters.
Board: A 6 x 7 grid (six rows, seven columns).
Rules: The first player places an 'X' in the bottom-most square of any column. Subsequent players take turns placing their mark in the lowest available square of any column.
Winning Condition: The first player to get four of their marks in a row horizontally, vertically, or diagonally wins.



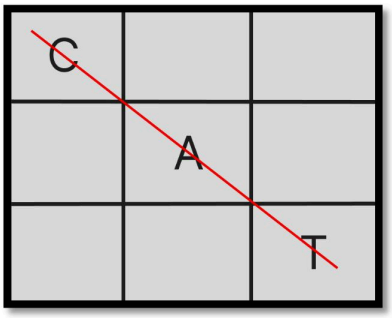
3. 5 x 5 Tic Tac Toe

Introduction: A variant played on a larger 5x5 grid where the goal is to have the most three-in-a-row sequences.
Board: A 5x5 grid.
Rules: Players take turns placing their mark in an empty square until only one square remains empty. This results in a total of 24 moves (12 for each player).
Winning Condition: The player who creates the most three-in-a-row sequences wins.



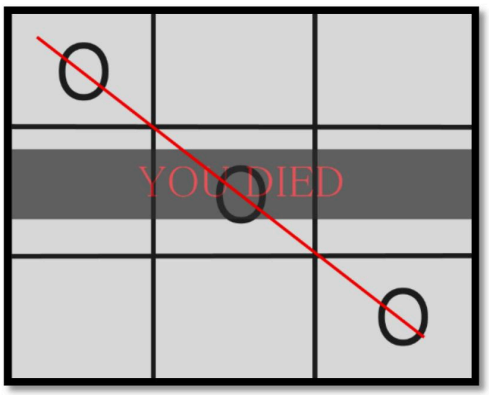
4. Word Tic-tac-toe

Introduction: A version where players place letters on the board to form valid three-letter words.
Board: A 3x3 grid.
Rules: Players take turns placing one letter on the board, attempting to form a valid word. Players can build upon letters already on the board.
Winning Condition: The first player to form a valid three-letter word horizontally, vertically, or diagonally wins. The game is a draw if the board is filled without a valid word being formed.
Additional Details: A file named dic.txt containing all valid 3-letter words will be provided.



5. Misère Tic Tac Toe

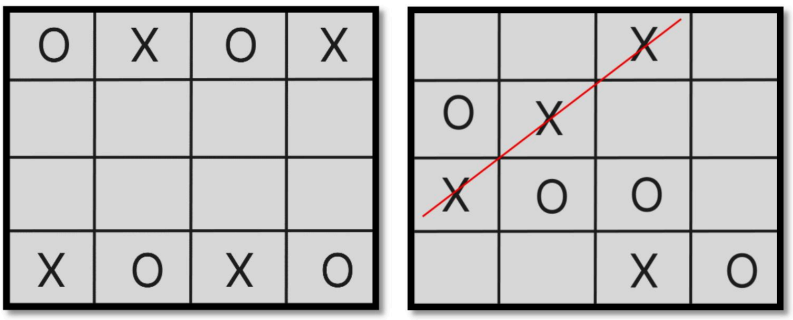
Introduction: Also known as Inverse Tic-Tac-Toe, the objective is to force your opponent to get three-in-a- row.
Board: A standard 3x3 grid.
Rules: The goal is to avoid placing three of your marks in a row, column, or diagonal.
Winning Condition: A player loses if they complete a line of three of their own marks. If the board is filled with no player achieving three-in-a-row, the game is a draw.



6. 4 x 4 Tic-Tac-Toe

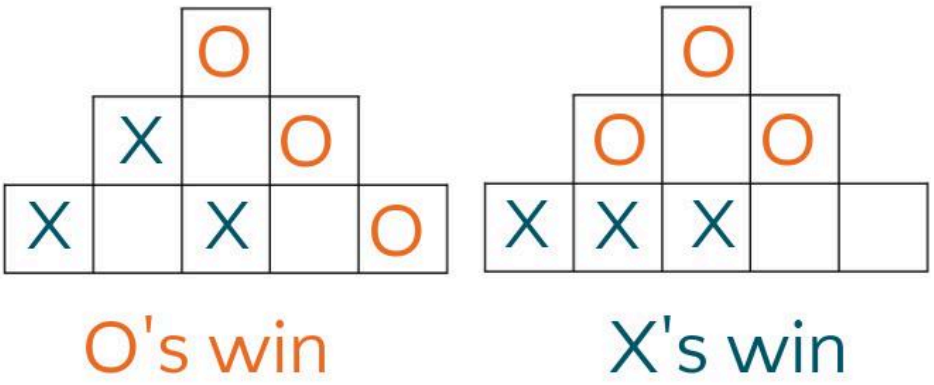
Introduction: An extended version on a larger board where players move their tokens to create a three-in-a-row alignment.

Board: A 4x4 grid. Each player starts with four tokens placed in specific starting positions.
Rules: Players take turns moving one of their tokens to an adjacent (horizontal or vertical) empty square. Tokens cannot move diagonally or jump over other tokens.
Winning Condition: The first player to align three of their tokens in a row, column, or diagonal wins.



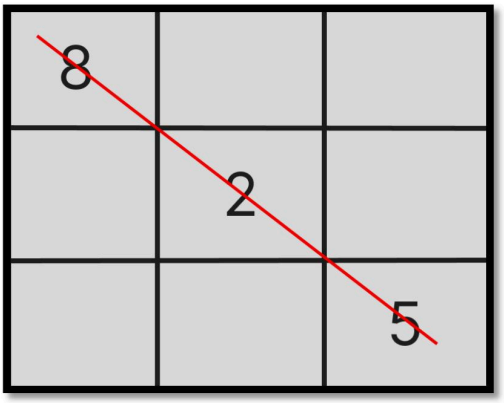
7. Pyramid Tic-Tac-Toe

Introduction: A Tic-Tac-Toe variant played on a pyramid-shaped game board.
Board: A pyramid structure with a base of five squares, a middle row of three, and one square at the top.
Rules: Players take turns placing their 'X' or 'O' marks in empty squares.
Winning Condition: The first player to align three of their marks horizontally, vertically, or diagonally wins.



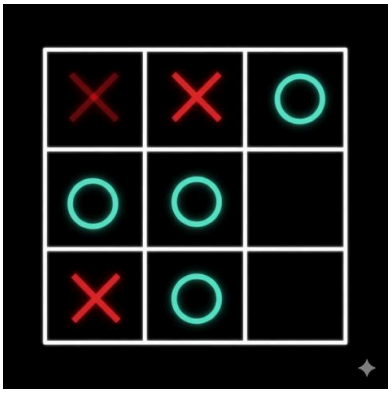
8. Numerical Tic-Tac-Toe

Introduction: A mathematical variant where players use numbers with the objective of making a line sum to 15.
Board: A 3x3 grid.
Rules: Player 1 uses odd numbers (1, 3, 5, 7, 9) and Player 2 uses even numbers (2, 4, 6, 8). Players alternate placing one number in an empty cell, and each number can only be used once.
Winning Condition: A player wins by placing three numbers in a row, column, or diagonal that add up to exactly 15. If all cells are filled and no line sums to 15, the game is a draw.



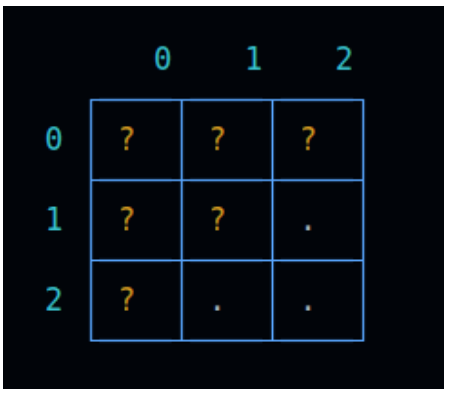
9.Infinity Tic-Tac-Toe

Introduction: A game on a 3x3 grid where the oldest moves are periodically removed from the board.
Board: A 3x3 grid.
Rules: After every three moves, the oldest mark on the board disappears.
Winning Condition: The first player to align three marks in a row before any of those marks vanish wins the game.



10.Memory Tic-Tac-Toe

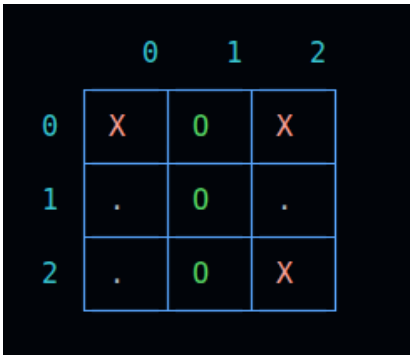
Introduction: A variant where marks are hidden after being placed, testing the players' memory.
Board: A 3x3 grid.
Rules: When a player places a mark, it is immediately hidden from view. Players must remember the positions of all marks to plan their strategy.
Winning Condition: The first player to align three of their hidden marks in a row wins. The game is a draw if the board fills without a winner.



11.Standard Tic-Tac-Toe (X-O)

Introduction: Standard Tic-Tac-Toe, also known as X-O, is a classic two-player game where players compete to align their marks in a straight line on a small grid.

Board: The game is played on a 3×3 grid consisting of nine cells.

Rules: Two players take turns placing their marks on the board. One player uses **X** and the other uses **O**. A player can place their mark only in an empty cell, and once placed, it cannot be moved or removed.

Winning Condition: The first player to align three of their marks in a row, column, or diagonal wins the game. If all cells are filled and no player achieves this, the game ends in a draw.